

# Derivability Classes of Optimal Reduction

## The Derivability Spectrum from Proof to Measurement

**Registry:** [\[HOWL-MATH-20-2026\]](#)

**Series Path:** [\[HOWL-INFO-11-2026\]](#) → [\[HOWL-INFO-12-2026\]](#) → [\[HOWL-INFO-13-2026\]](#) → [\[HOWL-MATH-14-2026\]](#) → [\[HOWL-MATH-15-2026\]](#) → [\[HOWL-MATH-16-2026\]](#) → [\[HOWL-MATH-17-2026\]](#) → [\[HOWL-MATH-18-2026\]](#) → [\[HOWL-MATH-19-2026\]](#) → [\[HOWL-MATH-20-2026\]](#)

**Date:** June 2026

**DOI:** 10.5281/zenodo.20630720

**Domain:** Information Processing Theory / Applied Mathematics / Computability Theory

**AI Usage Disclosure:** Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

---

### 1. Two Floors

A sorting algorithm has a provable minimum. No comparison-based sort can order  $N$  elements in fewer than  $\lceil \log_2(N!) \rceil$  comparisons. This is not an empirical finding. Nobody timed a thousand sorting algorithms and reported the fastest. It is a mathematical theorem derived from the task's information-theoretic structure:  $N!$  possible orderings exist, each comparison eliminates at most half, and therefore  $\log_2(N!)$  comparisons are necessary regardless of how clever the algorithm is. The floor is known before anyone sorts anything.

An emergency physician diagnosing chest pain has no such theorem. The best physicians reach a correct diagnosis in five to eight operations — a focused history question, a targeted examination maneuver, a key lab result, a pattern recognition, a confirmation. The worst take forty to sixty operations, methodically checking every possibility. The best observed performance is the only estimate of the floor. Maybe four operations suffice. Maybe three. Maybe the current best is already optimal. Nobody can prove it either way because no structural argument connects the task's properties to a minimum operation count.

These two cases represent opposite ends of a spectrum. At one end, the minimum number of operations for a task is derivable from the task's structure — you can prove the floor before anyone performs the task. At the other end, the minimum is known only from observation — you measure the best performers and call that the estimate. Between these ends lies territory where partial structural arguments constrain the minimum without determining it exactly.

This paper classifies tasks by where on this spectrum their minimum operation count lives, identifies the structural properties that determine placement, and examines whether the

classification has a hierarchy analogous to the complexity classes of theoretical computer science.

The vocabulary is small and builds in order. Processing is what any system does when it must act on information — a CPU executing instructions, a physician diagnosing, a pilot navigating, a developer debugging. The unit of processing is the **op**: one irreducible transformation by one processor. Processing entropy is the op count a specific processor requires for a specific task. Through repetition under consistent conditions, processing entropy decreases toward zero as the processing chain dissolves into structure that produces correct results without consuming the processor's scarce sequential pipeline.

Before dissolution, there is a floor: the **optimal reduction**  $R$ , *the minimum number of correct ops any competent processor requires for reliable execution of a given task*. Below  $R$ , the processor is not dissolved — it is operating without sufficient verification. Above  $R$ , *there is measurable inefficiency that practice can eliminate*.  $R$  is the boundary between competence and waste, and knowing its value is fundamental to training design, performance assessment, and efficiency engineering.

Prior work in this series defined  $R^*$ , used it to measure dissolution progress, and noted that it is provable in some domains and only empirical in others. This paper asks the structural question: what determines which?

---

## 2. Three Classes

Every task in every domain has an  $R^*$  — a minimum correct operation count. The task either has structure that makes  $R^*$  formally derivable, or it doesn't. The classification has three natural divisions based on what can be known about  $R^*$  from the task's properties.

**Class P: Provable.**  $R^*$  is derivable from the task's formal structure by logical argument. A proof exists — or can be constructed — that no correct execution uses fewer operations. The proof may be constructive, exhibiting a method that achieves  $R$ , *or existential, proving that fewer operations leave the processor unable to distinguish inputs that require different outputs*. The defining characteristic:  $R$  is known with certainty, as an exact value, before any processor executes the task. No amount of expert observation can revise it. The minimum is a theorem.

**Class B: Boundable.**  $R^*$  cannot be derived exactly, but structural arguments establish provable bounds. A lower bound  $R_{\text{lower}}$  is proven: no correct execution uses fewer operations. An upper bound  $R_{\text{upper}}$  is established: a known method achieves this count with correct results. The true  $R^*$  lies somewhere between:  $R_{\text{lower}} \leq R^* \leq R_{\text{upper}}$ . The gap between bounds may be narrow ( $R^*$  nearly pinned) or wide (substantial uncertainty persists). The defining characteristic:  $R^*$  is known to lie within a range with certainty, but the exact value remains undetermined. The gap reflects genuine structural uncertainty about the task, not merely incomplete analysis.

**Class E: Empirical.** No provable bound on  $R^*$  is available from the task's structure. The best current estimate of  $R^*$  is the performance of the best observed processor. If someone

demonstrates a lower operation count with correct results, the estimate revises downward. If no one improves on the current best, the estimate remains. The defining characteristic:  $R^*$  is known only from measurement and subject to revision upon new evidence. There is no structural argument establishing that the current best is optimal or even approximately optimal. The floor is a record, not a theorem.

The classification applies per-task, not per-domain. A single domain may contain tasks in all three classes. Mathematics includes tasks with provable minimum step counts (solving a specific equation type), tasks with proven bounds but unknown exact minimums (certain optimization problems), and tasks with purely empirical floors (finding the shortest proof of a novel theorem). Medicine includes a few algorithmic protocols with provable minimums (Class P), some diagnostic criteria with computable minimum checks (Class B), and the vast majority of clinical reasoning tasks with empirical floors only (Class E).

---

### 3. Class P — The Provable Floor

What structural properties make  $R^*$  provable? Examining tasks across domains where  $R^*$  has been formally established reveals four properties that, when present together, enable proof.

**Sorting by comparison.**  $R^* = \lceil \log_2(N!) \rceil$  comparisons. The input is a sequence of  $N$  elements from a totally ordered set. The output is the sorted permutation. Each comparison yields one bit of information (element  $A$  is greater or less than element  $B$ ). There are  $N!$  possible input orderings. Distinguishing among  $N!$  possibilities requires at least  $\log_2(N!)$  binary distinctions. Therefore at least  $\lceil \log_2(N!) \rceil$  comparisons are necessary. The proof follows from three structural facts: the input space is enumerable, each operation extracts a bounded amount of information, and the total information requirement is computable.

**Search in a sorted array.**  $R^* = \lceil \log_2(N) \rceil$  comparisons. Same structure:  $N$  possible positions for the target, each comparison eliminates at most half,  $\log_2(N)$  comparisons necessary. Binary search achieves this, so the lower bound is tight.

**Graph connectivity verification.**  $R^* = |E|$  edge examinations in the worst case. To verify that a graph is connected, every edge must be examined at least once, because any unexamined edge could be the one whose removal disconnects the graph. An adversary can always construct an input where the last examined edge determines the answer. The proof follows from the adversary argument: a processor that skips any edge can be fooled.

**Minimum spanning tree.**  $R^* = \Omega(E \log E)$  for comparison-based algorithms. Similar to sorting: the algorithm must distinguish between possible MSTs, and the information-theoretic argument bounds the minimum comparisons. For specific graph structures, tighter bounds exist.

**Parity checking.**  $R^* = N$  for determining parity of  $N$  bits. Every bit must be read — any unread bit could flip the answer. The proof is a one-line adversary argument.

These examples share four structural properties.

**Property P1: Enumerable input space.** The set of possible inputs is formally characterizable. You can count the inputs ( $N!$  permutations,  $N$  positions,  $2^N$  bit strings) or compute their information content (entropy of the input distribution). The input is not “whatever the world presents” but a member of a well-defined set.

**Property P2: Decidable correctness.** Given an input and an output, whether the output is correct is decidable — verifiable by a finite procedure. The sorted permutation is verifiable by checking adjacent pairs. Graph connectivity is verifiable by BFS/DFS. Parity is verifiable by XOR. There is no ambiguity about what constitutes a correct result.

**Property P3: Bounded information per operation.** Each operation extracts or transforms a quantifiable amount of information. A comparison extracts one bit. An edge examination reveals one edge’s presence or absence. When the information yield per operation is bounded and the total information requirement is known, the minimum operation count follows by division.

**Property P4: Constructible adversary.** A worst case can be explicitly constructed that forces any correct processor to use at least  $R^*$  operations. The adversary constructs inputs that are maximally unhelpful — making each operation yield as little useful information as possible, forcing the processor to use the maximum number of operations. Without the adversary argument, a lower bound proof must rely on other techniques (information theory, counting, reduction), but the adversary is the most common and often the most intuitive.

When all four properties hold simultaneously,  $R^*$  is provable. The proof typically has the form: the input space has information content  $I$ , each operation extracts at most  $b$  bits, therefore at least  $I/b$  operations are necessary. Or: an adversary can force the processor into at least  $R^*$  operations regardless of strategy. The structural properties make the proof possible; without them, the proof framework has no foundation to build on.

---

#### 4. Class B — The Bounded Floor

Between provability and pure empiricism lies a substantial territory where structural arguments constrain  $R^*$  without determining it exactly.

**NP-hard optimization.** The traveling salesman problem on  $N$  cities has no known polynomial-time exact solution. But bounds exist in both directions. The lower bound: the optimal tour length can be computed (by exponential-time exact algorithms), so  $R^*$  for “find the optimal tour” is exactly known in principle — the difficulty is computational cost, not derivability. For “find a good tour in polynomial time,” Christofides’ algorithm guarantees a tour within  $1.5 \times$  optimal on metric instances. The  $R^*$  for the approximate version is bounded: at least the cost of reading the input ( $\Omega(N^2)$  for distance matrix), at most the cost of Christofides’ algorithm ( $O(N^3)$ ). The gap between bounds is the gap between what must be done (read the input) and what the best known method does.

**Numerical integration.** Computing a definite integral to accuracy  $\epsilon$  requires some minimum number of function evaluations. For functions with  $k$  continuous derivatives, the

optimal convergence rate is  $O(\epsilon^{-1/k})$  evaluations — proven by information-based complexity theory. The lower bound is structural: functions with limited smoothness require a minimum number of samples to constrain the integral within  $\epsilon$ . The upper bound is achieved by optimal quadrature rules. The bounds are tight for specific smoothness classes but the smoothness of a given function may itself be unknown, introducing a gap between the structural bound and the achievable bound for a specific instance.

**Diagnostic criteria checking.** Some medical diagnoses have formal criteria: a defined list of findings that must be present or absent. Rheumatoid arthritis requires meeting four of seven American College of Rheumatology criteria. The lower bound: at least four findings must be verified (you cannot confirm four positives without checking at least four). The upper bound: check all seven (guarantees classification regardless of which four are positive). If the physician knows the base rates and correlations between findings, an optimal checking order exists that minimizes expected operations — and this expected  $R^*$  is computable from the base rates. But the optimal order depends on the specific patient's presentation, creating a gap between the population-level  $R^*$  (computable) and the individual-patient  $R^*$  (not computable until the findings are observed).

**Error-correcting codes.** Shannon's channel coding theorem establishes that reliable communication at rate  $R$  is possible if and only if  $R < C$  (channel capacity). The minimum number of operations to encode and decode at rate  $C$  is bounded: at least as many operations as bits processed, at most the complexity of the best known code for that channel. The gap between bounds has narrowed over decades (from Reed-Solomon to turbo codes to LDPC to polar codes) but for general channels, the exact  $R^*$  for encoding/decoding at capacity remains open.

**Signal reconstruction.** Compressed sensing establishes that a  $k$ -sparse signal of length  $N$  can be reconstructed from  $O(k \log(N/k))$  measurements — far fewer than  $N$ . The lower bound is  $\Omega(k)$  (must acquire at least as many measurements as unknowns). The upper bound is the number of measurements required by the best known recovery algorithm with the best known measurement matrix. The gap depends on the signal's structure and the measurement design.

The structural properties of Class B tasks:

**Property B1: Partially formal specification.** The task has formal properties — smoothness class, criterion count, graph structure, sparsity — that structural arguments can exploit. But the full task is not purely formal. Some aspect resists exact characterization: the specific input's difficulty, the interaction between formal structure and instance-specific features, or the optimal strategy within the formal constraints.

**Property B2: Bound-establishing techniques available.** Proof techniques exist that establish floors or ceilings. Information theory gives lower bounds. Known algorithms give upper bounds. Counting arguments, reduction, or adversary methods may contribute. But the techniques available for the lower bound and the techniques available for the upper bound don't converge to the same value.

**Property B3: The gap has structural significance.** The gap between lower and upper bounds is not merely a reflection of incomplete analysis. It persists because different

structural aspects of the task contribute to each bound. Closing the gap requires resolving a structural question: Is there a better algorithm? Is the lower bound too loose? Does the task have hidden structure that neither bound exploits? These questions may connect to the deepest open problems in their respective fields — including, in the computational case, the P versus NP problem, which is essentially the question of whether a large class of B-gaps can be closed.

---

## 5. Class E — The Empirical Floor

At the far end of the spectrum,  $R^*$  is known only from observation. No structural argument bounds it. The best performers define the floor, and the floor is subject to revision whenever someone performs better.

**General medical diagnosis.** A physician encountering a patient with undifferentiated symptoms — fatigue, weight loss, intermittent fever — must navigate a large differential diagnosis space. The best diagnosticians reach the correct diagnosis in five to eight operations for common presentations: a targeted history question that narrows the space dramatically, a physical finding that confirms or eliminates a major category, a laboratory result that discriminates between remaining candidates, and a synthesis that commits to the working diagnosis. But no structural argument proves this is the minimum. The symptom-to-diagnosis mapping is learned from thousands of cases. Different physicians learn different mappings. Some happen to be shorter. The floor is the shortest observed chain that reliably produces correct diagnoses. Whether a shorter reliable chain exists is unknown.

**Tactical combat decisions.** A fighter pilot classifying a radar contact as hostile or friendly integrates radar cross-section, speed, altitude, heading, IFF response, tactical context, and rules of engagement into a single classification. The best pilots accomplish this in three to four conscious operations. The worst take twelve to fifteen. No formal argument establishes the minimum because the input space (all possible radar presentations in all possible tactical contexts) is not formally characterizable, correctness is context-dependent (the same contact is hostile in one scenario and friendly in another), and the information content per operation is not bounded (a single radar blip may convey more information to an experienced pilot than to a novice because the experienced pilot's decomposition of the radar signature is richer).

**Creative mathematical proof.** A mathematician seeking a proof of a novel theorem has no bound on the minimum proof length. The search space is the space of all valid deductions from the axioms, and the minimum path from axioms to theorem through that space is unknown until the proof is discovered. After discovery, the shortest known proof provides an upper bound on  $R^*$ , and the proof must have at least one step (trivial lower bound), but the gap between these is typically enormous. For some theorems, astonishingly short proofs exist that were discovered decades after the first long proofs — Erdős's concept of "The Book" proof, the most elegant possible, has no structural characterization that would allow its length to be determined in advance.

**Culinary preparation.** A chef preparing a complex dish operates under physical constraints (must heat before caramelization, must emulsify before emulsion forms, must rest meat before carving) that establish weak lower bounds. But the overall preparation  $R^*$  — the minimum operations for the target quality — is empirical. Different chefs achieve comparable quality with different operation counts. Some have discovered optimizations others haven't (combining steps, parallelizing preparations, eliminating unnecessary intermediate checks). The floor is the best observed practice, revisable upon innovation.

**Software debugging (novel bugs).** Debugging a bug never previously encountered in an unfamiliar codebase has no structural  $R^*$ . The developer must explore the codebase, form hypotheses, test them, and locate the fault. The minimum operations depend on the codebase's structure, the bug's nature, and the developer's existing dissolved patterns for similar codebases. Different developers find the same bug in vastly different operation counts. The floor is the fastest observed resolution, subject to revision when a developer finds a shorter path.

The structural properties — or rather their absence — that characterize Class E:

**Property E1: Non-enumerable input space.** The input is not a member of a well-defined formal set. It is a pattern in a high-dimensional space — symptoms, radar returns, sensory data, social context, codebase state — without a finite enumeration of possible inputs or a computable information content. You cannot count the possible inputs because the input space is not formally bounded.

**Property E2: Graded correctness.** Correctness is not binary. A diagnosis can be more or less accurate. A tactical classification can be more or less appropriate. A proof can be more or less elegant. Without a crisp boundary between correct and incorrect outputs, the adversary argument framework (which requires distinguishing correct from incorrect) has no foundation. The notion of “minimum operations for a correct result” is blurred when correctness itself is a spectrum.

**Property E3: Learned mapping.** The task's input-to-output relationship is not specified by formal rules but learned from experience. Different processors learn different mappings from the same examples. The  $R^*$  for a learned mapping depends on the mapping's structure, which is itself not formally characterized — it emerged from training, not from specification.

**Property E4: Processor variability dominates.** Different processors achieve different  $R^*$  estimates not because the task has multiple correct solutions but because each processor has learned a different reduction chain. The variation in observed operation counts reflects variation in learned processing paths, not variation in task structure. This makes the floor a statistical property of the processor population rather than a structural property of the task.

## 6. The Structural Properties That Determine Class

The four properties of Class P (enumerable input, decidable correctness, bounded information per operation, constructible adversary) and the four anti-properties of Class E (non-enumerable input, graded correctness, learned mapping, processor-dominated variability) define the classification. Class B occupies the territory where some but not all Class P properties hold.

Arrange the properties as a checklist:

Does the task have a formally enumerable input space? If yes, information-theoretic lower bounds are possible. If no, lower bounds must come from weaker structural arguments or not at all.

Is correctness decidable — can you verify a result as correct or incorrect by finite procedure? If yes, adversary arguments can establish that fewer operations leave correctness unverifiable. If no, the notion of “minimum correct operations” is inherently imprecise.

Is the information yield per operation bounded and quantifiable? If yes, dividing total information requirement by per-operation yield gives a floor. If no, some operations may yield unpredictably more information than others (the expert’s single glance that extracts a diagnosis versus the novice’s ten detailed examinations that extract less).

Can a worst case be constructed that forces maximum operations? If yes, the lower bound is tight against all possible strategies. If no, the lower bound may depend on average-case or distributional assumptions that weaken the proof.

Class P tasks satisfy all four. Class E tasks satisfy none. Class B tasks satisfy some — and the specific subset of satisfied properties determines which proof techniques are available and how tight the resulting bounds can be.

A task satisfying only P1 (enumerable input) but not P2 (decidable correctness) — for instance, evaluating the aesthetic quality of  $N$  images — has a formally characterizable input space but no crisp correctness criterion. An information-theoretic argument can bound the operations needed to “process” the inputs but not the operations needed to produce a “correct” output, because correctness is undefined. This task falls in Class E despite having formal input structure.

A task satisfying P1 and P2 but not P3 (bounded information per operation) — for instance, expert-aided diagnosis where a single experienced observation may extract arbitrarily more information than a novice’s observation — has formal inputs and decidable correctness but no bound on per-operation information yield. A lower bound can be established in terms of total information requirement but not in terms of operation count, because the operation count depends on the processor’s dissolution state. This task falls in Class B: the total information requirement is a structural bound, but the operation count depends on the processor.

This last case reveals something important:  **$R^*$  may depend on the processor’s dissolution state for the same task.** A dissolved expert extracts more information per operation than a novice. If  $R^*$  is defined as the minimum across all possible processors (including



hypothetical optimal ones), it may be lower than what any existing processor achieves. If  $R^*$  is defined as the minimum for a given competence level (a specific dissolution state), it is processor-relative. The definition of  $R^*$  interacts with the classification: for Class P tasks,  $R^*$  is processor-independent (it depends on the task's information structure, not on who performs it). For Class E tasks,  $R^*$  may be inherently processor-relative.

---

## 7. The Hierarchy

The three classes form a hierarchy with a definite containment structure and a suggestive parallel to computational complexity theory.

**P  $\subset$  B.** Every provable floor is trivially a bounded floor where the gap between lower and upper bounds is zero. A Class P result is a Class B result with a tight bound. Class P is the special case of Class B where complete structural information is available.

**B contains results that are not in P.** Tasks with genuine bound gaps — where the lower bound proof technique and the upper bound construction technique don't converge — are in B but not in P. Whether the gap can eventually be closed (moving the task to P) or is inherent (the task is fundamentally in B–P) is, in many cases, an open question. For computational tasks, this connects to the P versus NP problem: if  $P \neq NP$ , then NP-hard tasks are permanently in B–P for polynomial-time computations.

**E contains results that are not in B.** Tasks with no structural bounds at all — where neither lower nor upper bound proofs are available from the task's structure — are in E but not in B. The question of whether any task is permanently in E (no structural bound will ever be found) or merely currently in E (structural analysis hasn't been done yet) is itself often unanswerable.

The hierarchy has a directional property: tasks can move toward P (gaining structural analysis) but should not move away from it (a proven bound doesn't become unproven). The movement is:

$E \rightarrow B$ : discovery of a structural bound that didn't previously exist.

$B \rightarrow P$ : closing the gap between known bounds.

$B \rightarrow P$  via separate convergence: improving the lower bound, improving the upper bound, or both, until they meet.

The parallel to computational complexity classes is structural:

**Derivability P  $\leftrightarrow$  Complexity P.** The answer ( $R^*$ ) is efficiently derivable from the task's structure. You can compute the floor.

**Derivability B  $\leftrightarrow$  Complexity NP.** The answer is bounded but not efficiently derivable. You can verify a claimed floor (run the proposed method, check if it's correct, count its operations) but you cannot efficiently derive the minimum. Bounds constrain  $R^*$  without determining it.

**Derivability  $E \leftrightarrow$  Complexity Undecidable.** No general structural method derives  $R^*$ . The floor is not merely hard to compute — it is structurally inaccessible from the task description. Only empirical observation provides estimates.

The parallel is suggestive but imperfect. Complexity classes describe worst-case resource requirements for computation. Derivability classes describe what can be known about optimal processing from structural analysis. The former is about the cost of solving problems. The latter is about the cost of knowing the cost. They operate at different meta-levels: complexity theory asks “how hard is the task?” while derivability theory asks “how hard is it to know how hard the task is?”

The imperfection runs deeper. Complexity classes apply to infinite families of instances (all graphs, all Boolean formulas). Derivability classes apply to individual tasks or task types. A sorting task has a provable  $R^*$  for all input sizes simultaneously (one theorem covers all  $N$ ). A medical diagnosis task has an empirical  $R^*$  that may differ for each presentation. The universality of complexity results (one proof covering all instances) versus the instance-specificity of many  $R^*$  estimates is a structural difference between the two classification systems.

---

## 8. Class Transitions

Tasks do not permanently belong to a class. Structural discoveries can move a task toward greater derivability. Examining historical transitions reveals what triggers the movement and what characterizes the discovery.

**Disease diagnosis:  $E \rightarrow B$ .** Before specific biomarkers, diagnosing many conditions was purely empirical — the best clinicians’ performance was the only  $R^*$  estimate. The discovery of troponin as a biomarker for myocardial infarction introduced a formal criterion: if troponin exceeds a threshold, myocardial injury is present. This created a structural lower bound:  $R^*$  for diagnosing MI must include at least one operation (measuring troponin). Combined with the clinical criteria (symptoms, ECG findings, biomarker), the minimum diagnostic operations gained a computable lower bound — the number of independent criteria that must be checked. The task moved from  $E$  (no structural bound) to  $B$  (lower bound from criteria count, upper bound from clinical protocol).

**Chess endgame:  $B \rightarrow P$ .** Before endgame tablebases, the minimum number of moves to checkmate from a given position was bounded: known heuristics gave upper bounds, and the requirement to make at least one move gave a trivial lower bound. After exhaustive backward computation of all positions with up to seven pieces, the exact minimum for each position is known. The task moved from  $B$  to  $P$  for positions covered by the tablebase. The transition was driven by exhaustive computation rather than structural insight — brute force rather than elegance, but the result is the same: a proven exact  $R^*$ .

**Protein structure prediction:  $E \rightarrow B$ .** Before computational structure prediction, the minimum operations to determine a protein’s 3D structure from its sequence was purely empirical — only X-ray crystallography or NMR provided answers, and the “operation

count” was the experimental procedure’s complexity. AlphaFold’s architecture established an upper bound (the model’s computational cost for a given sequence length). The physics of protein folding establishes a loose lower bound (must process at least the sequence, considering each residue’s interactions with its neighbors). The task moved from deep E (purely experimental) toward B (computational bounds exist). Whether the task can reach P (provable minimum computation for a given accuracy) depends on whether the protein folding problem has sufficient mathematical structure — an open question.

**Arithmetic:  $E \rightarrow P$  via notation.** Before positional notation with zero, arithmetic was performed through diverse ad hoc procedures — different methods for different scales, different tools for different operations. The operation count for addition of two N-digit numbers was empirically observed, varying by method and practitioner. Positional notation made the task formally structured: N-digit addition requires exactly N single-digit additions plus at most N carries, giving  $R^* = N$  to  $2N$  operations. The notation didn’t change the task — it formalized the task’s structure, making the previously empirical floor provably derivable.

**Sorting: always P.** Comparison-based sorting has been Class P since the information-theoretic argument was first articulated. The  $R^*$  was provable as soon as someone asked the question. Some tasks are born into Class P because their structure is inherently formal.

The common trigger for  $E \rightarrow B$  transitions: **discovery of formal structure within the task.** Biomarkers added formal criteria to an empirical process. Computational models added formal upper bounds to an experimental process. Notation formalized an ad hoc procedure. In each case, something that was previously informal (learned, practiced, varied across processors) became formal (specified, bounded, structural).

The common trigger for  $B \rightarrow P$  transitions: **closing the gap between bounds.** Tablebases computed the exact answer for covered positions. Better algorithms reduced upper bounds to meet lower bounds. New proof techniques tightened lower bounds to meet upper bounds. The gap between bounds represents what is not yet known; closing it represents complete structural understanding.

---

## 9. Domain Classification

Apply the classification to every domain examined throughout the prior series. The classification is per-task: each domain contains tasks across the spectrum.

**Computation.** Predominantly Class P with significant Class B representation. Sorting, searching, graph traversal, matrix operations, parity checking — all have proven  $R^*$  values from information-theoretic or adversary arguments. NP-hard optimization has Class B status (bounds exist, gap remains). Heuristic algorithm design for novel problems is Class E (no structural method determines the minimum operations for the heuristic itself — only experimentation reveals which heuristics work and how efficiently).

**Mathematics.** Full spectrum. Specific computations (evaluating a polynomial, solving a linear system, computing a determinant) are Class P — minimum operation counts

provable from algebraic complexity theory. Optimization problems and approximation tasks are often Class B (bounds from information-based complexity, gaps from open algorithmic questions). Creative proof search and conjecture resolution are Class E — no structural bound on minimum proof length for novel theorems. Notably, mathematics is the domain most likely to generate transitions from E and B toward P, because mathematical research is precisely the activity of finding formal structure in previously informal territory.

**Medicine.** Predominantly Class E with some Class B. Most clinical reasoning — differential diagnosis, treatment selection, prognosis estimation — is Class E. The input space (patient presentations) is not formally enumerable, correctness is graded (better and worse diagnoses, not binary right/wrong), and the processing mapping is learned from clinical experience. Diagnostic tasks with formal criteria (classification systems, scoring systems, biomarker thresholds) are Class B: the criteria establish lower bounds on minimum checks, known protocols establish upper bounds, and the gap reflects the uncertainty in optimal checking order. A small number of algorithmic clinical protocols (ACLS cardiac arrest algorithm, sepsis screening protocol) approach Class P: the protocol steps are formally specified, and minimality arguments based on the decision tree structure can establish the floor.

**Aviation.** Mixed. Procedural tasks (checklist execution, instrument cross-check sequences) are Class P or near-P: the minimum steps are derivable from the procedure's structure and the information each step provides. Navigation computations are Class P (minimum waypoints computable from geometry and accuracy constraints). Tactical decision-making (threat classification, engagement sequencing, emergency response) is Class E: the input space is not formally bounded, correctness is context-dependent, and expert performance is the only floor estimate.

**Software engineering.** Mixed. Bug types with known structural patterns have Class B status: the minimum diagnostic steps are bounded by the pattern's structure (a null pointer dereference requires at least identifying the null reference and the dereference point — two operations minimum; the upper bound is the best known debugging technique for that pattern). Novel bugs in unfamiliar code are Class E: no structural argument bounds the search. Algorithmic implementations have Class P status inherited from the algorithm's complexity analysis. Code review and design assessment are Class E: no formal minimum for evaluating code quality.

**Manufacturing.** Mostly Class B. Physical constraints establish lower bounds that are often tight: assembling a component requires at least the operations to position each part and fasten it. The minimum is bounded below by physical necessity and above by the best known assembly sequence. Jig and fixture design can provably eliminate specific operations. The gap between physical minimum and achieved practice is typically small and closing — manufacturing is a domain where Class B tasks frequently approach Class P through process engineering.

**Cooking.** Predominantly Class E with some Class B elements. Physical and chemical constraints create weak lower bounds: must heat above a temperature for a duration, must combine ingredients in specific orders for chemical reactions. But the overall preparation

$R^*$  — the minimum operations for target quality — is empirical. The best chefs define the floor through practice, not through structural derivation.

**Air traffic control.** Mixed. Separation assurance computations are Class P or B: the minimum checks for safe separation are derivable from the geometry and closure rates of conflicting aircraft. Sector management under dynamic traffic is Class E: the minimum operations for optimal flow management are empirical, varying by controller experience and traffic complexity.

**Customer support.** Mostly Class E for general troubleshooting. Some Class B for known issue types with structured diagnostic trees (the minimum steps through the tree are computable, the upper bound is the full tree traversal, the gap reflects unknown shortcut paths). Known-issue resolution with documented solutions approaches Class P when the solution path is fully specified.

---

## 10. Implications for Training and Assessment

The derivability class of  $R^*$  has direct, practical consequences for how processors should be trained and assessed, because the class determines what targets exist and how progress toward them can be measured.

**Training for Class P tasks.** The floor is known. Training can target it precisely: the curriculum is designed to bring the processor's operation count down to  $R^*$  and then dissolve the chain to zero. Every operation above  $R^*$  is identifiable waste — not stylistic variation but provable inefficiency. Training materials can be built to eliminate exactly the unnecessary operations because the necessary ones are known. The dissolution curve has a defined endpoint that the trainer and trainee both know in advance.

Assessment for Class P tasks is unambiguous. The processor either achieves  $R^*$  or is above it, and the distance is meaningful — an operation count of  $R^* + 3$  means exactly three unnecessary operations remain. Certification can be rigorous: demonstrate  $R^*$  or better. The standard is objective, structural, and non-negotiable.

**Training for Class B tasks.** The floor is bounded. Training can target the upper bound (bring the processor's count to the best known method's cost) and aspire toward the lower bound (which may or may not be achievable). The gap between bounds is the uncertainty in training targets — the trainer cannot be certain whether the current best method is optimal. Training materials target the best known method, which may itself contain unnecessary operations that future discoveries will eliminate.

Assessment for Class B tasks measures position within the bounds. A processor at  $R_{\text{upper}}$  is performing at the best known level. A processor between  $R_{\text{upper}}$  and some higher count has measurable improvement available by adopting the best known method. Whether the processor can get below  $R_{\text{upper}}$  is unknown — it depends on whether  $R^*$  is closer to  $R_{\text{lower}}$  than  $R_{\text{upper}}$ .

**Training for Class E tasks.** The floor is unknown. Training targets observed best performance, which may itself be far from the true  $R^*$ . The trainer cannot guarantee that

their methods are optimal because nobody can. Training materials codify current best practice, which contains whatever inefficiencies the best practitioners happen to have — including ones nobody has yet identified as inefficiencies.

Assessment for Class E tasks is relative rather than absolute. The processor's count is compared to the population distribution, not to a known floor. Being in the top 5% of performers is the closest available analog to “near optimal,” but the top 5% might be far from  $R^*$  — there is no way to know. Progress is measured by improving relative standing or by beating the best observed performance, not by approaching a known limit.

This produces a counterintuitive observation about expertise: **experts in Class E domains don't know if they're optimal, and neither does anyone else.** The best surgeon, the best pilot, the best diagnostician — each defines the current empirical floor, but none can be certain they haven't included unnecessary operations that a future practitioner will eliminate. In Class P domains, the optimal performer can be verified as optimal. In Class E domains, the “optimal” performer is merely the best observed so far.

---

## 11. The Classification Procedure

Provide a mechanical procedure for classifying a task's  $R^*$  into P, B, or E. The procedure works by testing for the structural properties identified in Sections 3 through 5, in a specific order designed to catch the earliest definitive signal.

**Step 1: Formal input characterization.** Can you formally define the space of possible inputs to this task? Can you enumerate the inputs, compute their count, or characterize their information content? “All permutations of  $N$  elements” is formal. “All possible patient presentations” is not. If the input space is formally characterizable, proceed to Step 2. If not, the task is **provisionally Class E**. The word “provisionally” acknowledges that formal characterization might be possible but hasn't been achieved yet.

**Step 2: Decidable correctness.** Given an input and a proposed output, can you determine whether the output is correct by a finite procedure? “Is this the sorted permutation?” is decidable. “Is this the correct diagnosis?” may not be — the true diagnosis may be uncertain, or correctness may be graded. If correctness is decidable, proceed to Step 3. If correctness is graded or subjective, the task is **Class E** (no crisp correctness criterion means no precise  $R^*$  definition).

**Step 3: Information-theoretic lower bound.** Can you compute the minimum information that must be extracted from the input to produce a correct output? If the input space has  $N$  possibilities and each operation extracts at most  $b$  bits, then  $\lceil \log_2(N)/b \rceil$  operations are necessary. If such a computation is possible, it establishes a lower bound on  $R^*$ . Proceed to Step 4. If the per-operation information yield is not boundable (varies by processor, context, or method), the task is **Class B** with a lower bound from Step 1's input characterization but without an information-theoretic floor.

**Step 4: Matching upper bound.** Does a known method achieve the lower bound from Step 3? If yes — the lower bound is achievable — then  $R^*$  equals the lower bound: **Class**

**P**. If no known method matches the lower bound but methods exist with quantifiable costs, the task is **Class B** with a computable gap. If no quantifiable upper bound exists (all known methods have uncharacterized costs), the task is **Class B** with open upper bound.

The procedure is conservative. It classifies toward E when in doubt. This is correct behavior: a task misclassified as “more derivable than it is” would generate false certainty about  $R^*$ , while a task misclassified as “less derivable than it is” merely indicates that structural analysis hasn’t been completed yet. False negatives (classifying P as B, or B as E) are harmless — they underestimate what is known. False positives (classifying E as P) would be dangerous — they would assert certainty that doesn’t exist.

The procedure is also recursive. Applying it to a new task may reveal that Step 1 or Step 3 requires answering sub-questions whose own classification is unknown. “Can you characterize the input space?” may require solving a characterization problem that is itself Class E. This recursion connects to the open problem of meta-derivability discussed in Section 13.

---

## 12. Dissolution Infrastructure and Class Membership

The derivability class of  $R^*$  determines what dissolution infrastructure can achieve and what it cannot.

For Class P tasks, dissolution infrastructure can be **provably optimal**. Since  $R^*$  is known, the infrastructure can be designed to guide the processor to exactly  $R^*$  operations and eliminate exactly the unnecessary ones. A checklist for a Class P procedural task contains exactly  $R^*$  items — no more (every item is necessary) and no fewer (every omitted item would leave a necessary operation unperformed). The checklist is provably complete and provably minimal.

For Class B tasks, dissolution infrastructure is **best-effort**. It targets the best known method (the upper bound  $R_{\text{upper}}$ ) and cannot guarantee that it’s not teaching unnecessary operations. The infrastructure is as good as current knowledge allows, but current knowledge may include operations that are actually unnecessary. A clinical checklist based on formal diagnostic criteria covers the known lower bound (all required checks) but may include checks that a future optimization will eliminate.

For Class E tasks, dissolution infrastructure is **empirically calibrated**. It codifies the best observed practice — the operations that the best practitioners perform — without knowing whether those operations are all necessary. The infrastructure may include operations that are actually unnecessary but that nobody has identified as such. It may also miss operations that could be necessary under conditions the best practitioners haven’t encountered.

This produces a practical prediction: **dissolution infrastructure for Class E tasks should be revised more frequently than for Class P tasks**. Class P infrastructure is provably optimal and needs revision only if the task changes. Class E infrastructure is

empirically optimal and needs revision whenever a practitioner discovers a shorter path. The revision frequency of infrastructure tracks the derivability class of the task it serves.

A second prediction: **the gap between expert and infrastructure-aided novice performance is smallest for Class P tasks and largest for Class E tasks.** For Class P tasks, the infrastructure encodes the provably optimal path — a novice following the infrastructure matches the expert’s  $R^*$ . For Class E tasks, the infrastructure encodes the best known path, which may be longer than the expert’s actual path (the expert has dissolved optimizations that the infrastructure hasn’t captured). The expert outperforms the infrastructure because their dissolved processing includes shortcuts that resist formalization.

---

### 13. Scope and Open Problems

This paper establishes three derivability classes for optimal reduction (Provable, Boundable, Empirical), identifies the structural properties that determine class membership, constructs a hierarchy with directional transitions, examines historical class transitions, classifies tasks across nine domains, derives implications for training and assessment, and provides a mechanical classification procedure.

The following remain open.

**Formal relationship to complexity classes.** The structural parallel between derivability classes and computational complexity classes (P, NP, Undecidable) is developed as an analogy. Whether a formal reduction or correspondence exists — proving that derivability P is equivalent to some complexity class, or that derivability E is equivalent to undecidability in some technical sense — would connect the processing framework to the deepest results in theoretical computer science. The connection through information-based complexity theory (which already classifies problems by the information needed to solve them) may provide the formal bridge.

**Class B gap dynamics.** For tasks in Class B, the gap between lower and upper bounds changes over time as better algorithms are discovered and better lower bound proofs are established. Is the rate of gap closure predictable from the task’s structural properties? Tasks where the gap has been narrowing steadily may be approaching Class P. Tasks where the gap has remained stable for decades may be fundamentally in B–P. Characterizing gap dynamics would help predict which Class B tasks are likely to become Class P in the foreseeable future.

**Dissolution curve dependence on class.** The prediction that Class P tasks dissolve faster than Class E tasks (because the known floor enables targeted training) is testable but untested. Measuring dissolution curves for tasks of known class, controlling for task difficulty and processor characteristics, would determine whether derivability class has a measurable effect on the speed and shape of dissolution.

**Meta-derivability.** The  $R^*$  of the process of determining  $R^*$  — how many operations does it take to classify a task? The classification procedure in Section 11 involves answering structural questions about the task, each of which has its own processing cost. For



some tasks, the meta-question (what class is this task's  $R^*$ ?) may itself be undecidable — there may be no finite procedure that determines whether a formal input characterization exists. This connects to Gödel's incompleteness results and the halting problem: some structural questions about a task's properties may be unanswerable within any fixed formal system.

**Subclasses within E.** Not all Class E tasks are equally opaque. Some have weak structural arguments that constrain  $R^*$  without providing numerical bounds (a diagnosis involving five organ systems must include at least one finding per system — a structural constraint, but not a numerical bound on total operations because the per-system finding cost varies). Whether the interior of Class E has a useful substructure — degrees of empirical-ness — is an open taxonomic question.

**Cross-domain  $R^*$  comparison.**  $R^* = 5$  ops in medicine and  $R^* = 5$  ops in software engineering. Are these comparable? The op durations differ (seconds versus minutes), the information content per op differs, and the formal properties differ. Whether  $R^*$  values are cross-domain comparable, and if so what normalization is required (duration-weighted? information-weighted? dissolution-curve-normalized?), is an open measurement question that connects to the series' broader goal of a universal processing theory.

---

## Appendix: Supporting Tables

### HOWL-MATH-20-2026

---

**Table A: Formal Definitions**

| Symbol  | Name               | Definition                                                                                            | Unit | Scope                                                                |
|---------|--------------------|-------------------------------------------------------------------------------------------------------|------|----------------------------------------------------------------------|
| $R^*$   | Optimal reduction  | Minimum number of correct ops any competent processor requires for reliable execution of a given task | ops  | Task-specific (Class P: exact; Class B: bounded; Class E: estimated) |
| R lower | Proven lower bound | Minimum ops established by structural argument; no correct execution uses fewer                       | ops  | Class B and P only                                                   |
| R upper | Proven upper bound | Op count achieved by best known correct method                                                        | ops  | Class B and P only                                                   |

| Symbol      | Name                       | Definition                                                                                                        | Unit | Scope                                                    |
|-------------|----------------------------|-------------------------------------------------------------------------------------------------------------------|------|----------------------------------------------------------|
| R empirical | Empirical floor estimate   | Lowest op count observed from any processor with correct results                                                  | ops  | All classes; only estimate available for Class E         |
| gap(task)   | Bound gap                  | $R_{\text{upper}} - R_{\text{lower}}$ ; structural uncertainty in $R^*$                                           | ops  | Class B only; zero for Class P; undefined for Class E    |
| Class P     | Provable                   | $R^*$ derivable from task structure by logical argument; exact value known with certainty                         | —    | Tasks with $P1 \wedge P2 \wedge P3 \wedge P4$            |
| Class B     | Boundable                  | $R^*$ constrained by provable bounds; exact value undetermined; $R_{\text{lower}} \leq R^* \leq R_{\text{upper}}$ | —    | Tasks with some but not all of P1–P4                     |
| Class E     | Empirical                  | $R^*$ known only from observation; no structural bound available; subject to revision                             | —    | Tasks lacking P1–P4                                      |
| P1          | Enumerable input space     | Set of possible inputs is formally characterizable; count or information content computable                       | —    | Structural property enabling lower bound proofs          |
| P2          | Decidable correctness      | Whether output is correct given input is determinable by finite procedure                                         | —    | Structural property enabling adversary arguments         |
| P3          | Bounded information per op | Each operation extracts or transforms a quantifiable, bounded amount of information                               | —    | Structural property enabling information-theoretic floor |

| Symbol | Name                    | Definition                                                                       | Unit | Scope                                           |
|--------|-------------------------|----------------------------------------------------------------------------------|------|-------------------------------------------------|
| P4     | Constructible adversary | Worst case constructible that forces any correct processor to use $\geq R^*$ ops | —    | Structural property enabling tight lower bounds |

**Table B: Class Comparison**

| Property                       | Class P (Provable)                                    | Class B (Boundable)                                                           | Class E (Empirical)                                   |
|--------------------------------|-------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------|
| R* status                      | Exact value known                                     | Constrained to range                                                          | Estimated from observation                            |
| Certainty                      | Mathematical proof                                    | Proven bounds with gap                                                        | Best observed; revisable                              |
| Source of knowledge            | Task structure (logical derivation)                   | Partial task structure (structural bounds + algorithmic upper bounds)         | Processor population (measurement of best performers) |
| Revision conditions            | Never (theorem is permanent)                          | Gap narrows with new algorithms or proofs; never widens                       | Revises downward when better performer observed       |
| Proof techniques               | Information-theoretic; adversary; counting; reduction | Same as P for lower bound; algorithmic analysis for upper bound; gap persists | None available for R* itself                          |
| Input space                    | Formally enumerable                                   | Partially formal                                                              | Not formally characterizable                          |
| Correctness criterion          | Decidable (binary)                                    | Decidable or partially decidable                                              | Graded or context-dependent                           |
| Information per op             | Bounded and quantifiable                              | Partially characterizable                                                     | Unbounded or processor-dependent                      |
| Adversary constructibility     | Yes                                                   | Partial (for lower bound)                                                     | No                                                    |
| Structural properties required | $P1 \wedge P2 \wedge P3 \wedge P4$                    | Some subset of P1–P4                                                          | None of P1–P4                                         |
| Training target                | Exact: train to R*                                    | Range: train to R upper, aspire to R lower                                    | Relative: train to best observed                      |

| Property                  | Class P (Provable)                                  | Class B (Boundable)                                           | Class E (Empirical)                                            |
|---------------------------|-----------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------|
| Assessment standard       | Absolute: distance from proven $R^*$                | Bounded: position within $[R \text{ lower}, R \text{ upper}]$ | Relative: percentile in population                             |
| Infrastructure optimality | Provably optimal (encodes exactly $R^*$ operations) | Best-effort (encodes $R$ upper; may include unnecessary ops)  | Empirically calibrated (encodes best practice; revision-prone) |

**Table C: Class P Examples**

| Task                   | Domain    | $R^*$<br>(proven)                                | Proof<br>Tech-<br>nique                                 | Input<br>Space (P1)                      | Correctness<br>(P2)                        | Info/Op<br>(P3)        | Adversary<br>(P4)                               |
|------------------------|-----------|--------------------------------------------------|---------------------------------------------------------|------------------------------------------|--------------------------------------------|------------------------|-------------------------------------------------|
| Comparison-based sort  | Computing | $\frac{1}{2} \log_2(N!)$<br>$\approx N \log_2 N$ | Information-theoretic: $N!$ orderings, 1 bit/comparison | $N!$ permutations                        | Is output sorted? Decidable in $O(N)$      | 1 bit per comparison   | Adversary maintains maximum remaining orderings |
| Search in sorted array | Computing | $\frac{1}{2} \log_2 N$                           | Information-theoretic: $N$ positions, 1 bit/comparison  | $N$ possible positions                   | Is target at position? Decidable in $O(1)$ | 1 bit per comparison   | Adversary maintains maximum remaining positions |
| Parity of $N$ bits     | Computing | $N$                                              | Adversary: any unread bit could flip answer             | $2^N$ bit strings                        | Is parity correct? Decidable in $O(1)$     | 1 bit per read         | Adversary flips unread bit                      |
| Graph connectivity     | Computing | $O(E)$ edge checks                               | Adversary: unexamined edge could disconnect             | All graphs on $V$ vertices and $E$ edges | Is graph connected? Decidable by BFS       | 1 edge per examination | Adversary removes unexamined edge               |

| Task                                                | R*<br>Domain(proven)                                                      | Proof<br>Tech-<br>nique                                                                  | Input<br>Space (P1)                               | Correctness<br>(P2)                               | Info/Op<br>(P3)                                    | Adversary<br>(P4)                                    |
|-----------------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------|---------------------------------------------------|----------------------------------------------------|------------------------------------------------------|
| Matrix<br>vector<br>mul-<br>ti-<br>ply              | Computational<br>$2N^2 \times N$                                          | Counting:<br>must<br>touch<br>each<br>matrix<br>entry at<br>least once                   | $N \times N$<br>matrices<br>$\times$<br>N-vectors | Is result<br>correct?<br>Decidable<br>in $O(N^2)$ | Bounded<br>by<br>arith-<br>metic<br>preci-<br>sion | Direct<br>counting<br>argument                       |
| Polynomial<br>eval-<br>ua-<br>tion                  | Computational<br>multiplica-<br>tions +<br>N addi-<br>tions<br>(Horner's) | Proven<br>optimal<br>for<br>general<br>polyno-<br>mial<br>degree N                       | Polynomial<br>coeffi-<br>cients +<br>point        | Is<br>evaluation<br>correct?<br>Decidable         | Fixed<br>per<br>arith-<br>metic<br>op              | Algebraic<br>lower<br>bound                          |
| N-<br>digit<br>ad-<br>di-<br>tion                   | Mathematics<br>$2N$ op-<br>erations<br>(addi-<br>tions +<br>carries)      | Structural:<br>must<br>process<br>each<br>digit;<br>carry<br>propaga-<br>tion<br>bounded | N-digit<br>number<br>pairs                        | Is sum<br>correct?<br>Decidable<br>in $O(N)$      | 1 digit<br>per op-<br>eration                      | Each<br>digit can<br>carry                           |
| Check-<br>pro-<br>to-<br>col<br>(med-<br>i-<br>cal) | Medicine<br>Number<br>of<br>check-<br>list<br>items                       | Structural:<br>each item<br>is an inde-<br>pendent<br>necessary<br>check                 | Defined<br>item set                               | All items<br>checked?<br>Decidable                | 1 item<br>per<br>check                             | Any<br>skipped<br>item<br>could be<br>the<br>failure |

**Table D: Class B Examples**

| Task                                                           | Domain            | R lower (proven)                                | R upper (best known)                                            | Gap                                                      | Gap Source                                                        | Path to Closing                                                  |
|----------------------------------------------------------------|-------------------|-------------------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------|
| Metric TSP (approximate)                                       | Computation       | $O(N^2)$ (read distance matrix)                 | $O(N^3)$ (Christofides $1.5 \times$ optimal)                    | $O(N^3)$ – $O(N^2)$                                      | Unknown whether sub-cubic $1.5 \times$ approximation exists       | Better approximation algorithms or tighter lower bounds          |
| Matrix multiplication                                          | Computation       | $O(N^2)$ (must read inputs)                     | $O(N^{2.371})$ (current best)                                   | $N^{2.371}$ – $N^2$                                      | Algebraic complexity theory; open whether $\omega = 2$ achievable | New algebraic techniques                                         |
| Numerical integration (smooth functions, accuracy $\epsilon$ ) | Mathematics       | $O(\epsilon^{-1/k})$ for $k$ -smooth functions  | $O(\epsilon^{-1/k})$ for optimal quadrature on known smoothness | Small for known smoothness; large for unknown smoothness | Function smoothness may be unknown                                | Adaptive methods; smoothness estimation                          |
| Compressed sensing recovery                                    | Signal processing | $\Omega(k)$ measurements (sparsity lower bound) | $O(k \log(N/k))$ measurements (RIP-based)                       | $\log(N/k)$ – $k$                                        | Measurement matrix design; signal structure                       | Better measurement matrices; structure-exploiting algorithms     |
| RA diagnosis (ACR criteria)                                    | Medicine          | 4 checks (minimum for 4-of-7 positive)          | 7 checks (verify all criteria)                                  | 3 checks                                                 | Correlation between criteria unknown per patient                  | Patient-specific correlation modeling; Bayesian optimal ordering |

| Task                            | Domain             | R lower (proven)                          | R upper (best known)                                          | Gap                                          | Gap Source                                                 | Path to Closing                                                                |
|---------------------------------|--------------------|-------------------------------------------|---------------------------------------------------------------|----------------------------------------------|------------------------------------------------------------|--------------------------------------------------------------------------------|
| Protein structure from sequence | Biology            | $\Omega(N)$ (must process sequence)       | $O(N^4)$ approximate (AlphaFold-class)                        | Very large                                   | Physics of folding not fully characterized computationally | Better structural models; proven folding lower bounds                          |
| Chess (general)                 | Game theory        | $\Omega(1)$ (must make at least one move) | Minimax depth $d$ with branching factor $b$ : $O(b^d)$        | Enormous                                     | Game tree too large for exhaustive analysis                | Exhaustive computation (infeasible for full game); better evaluation functions |
| Channel coding at capacity      | Information theory | $\Omega(N)$ (must process all bits)       | $O(N \log N)$ (polar codes)                                   | $O(N \log N)$ – $O(N)$                       | Encoding/decoding complexity of capacity-achieving codes   | Better code constructions                                                      |
| SAT (general)                   | Computation        | $\Omega(N)$ (must read formula)           | $O(2^N)$ (exhaustive search); better for structured instances | Exponential                                  | P vs NP                                                    | Resolve P vs NP; or prove stronger lower bounds                                |
| Minimum spanning tree           | Computation        | $O(E)$ (must examine edges)               | $O(E \alpha(V))$ (inverse Ackermann; nearly linear)           | Small ( $\alpha(V)$ grows incredibly slowly) | Whether $\Omega(E)$ is tight                               | Nearly closed; $\alpha(V)$ factor may or may not be eliminable                 |

**Table E: Class E Examples**

| Task                                  | R empirical (best Domain observed)                                     | Why Not Boundable                                                                                                                                  | Property Missing                                           | Variance Across Processors                                  |
|---------------------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|-------------------------------------------------------------|
| Undifferentiated chest pain diagnosis | Medicine<br>5–8 ops (expert)                                           | Symptom space not formally enumerable; correctness graded (working diagnosis, not binary); per-op information yield varies by physician experience | P1, P2, P3, P4 all absent                                  | 5–60 ops (expert to novice); $10 \times$ range              |
| Threat classification (combat)        | Aviation<br>3–4 conscious ops (top pilot)                              | Tactical situation space unbounded; correctness context-dependent; information per sensor check varies by pilot's dissolved pattern library        | P1, P2, P3 absent                                          | 3–15 ops; $5 \times$ range                                  |
| Novel theorem proof                   | Mathematics<br>Axioms per theorem; shortest known proof is R empirical | Proof space is infinite; minimum proof length undecidable in general; no adversary constructible against unknown search space                      | P1 partially present; P4 absent; meta-level undecidability | Unbounded; proofs range from trivial to fields-medal-worthy |



| Task                                 | R empirical (best Domain observed)          | Why Not Boundable                                                                                                                               | Property Missing | Variance Across Processors               |
|--------------------------------------|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|------------------|------------------------------------------|
| Novel bug in unfamiliar code-base    | Software5 ops (expert on familiar patterns) | Codebase state space not formally bounded; bug type unknown until found; diagnostic information per op varies by developer's dissolved patterns | P1, P3 absent    | 5–40 ops; $8 \times$ range               |
| Recipe optimization (target quality) | Cooking7 ops per dish (expert chef)         | Quality is graded not binary; ingredient interaction space not formally characterizable; physical constraints give only weak lower bounds       | P1, P2 absent    | 7–30 ops; $4 \times$ range               |
| Negotiation strategy                 | BusinessUnknown (no measurement standard)   | Outcome space unbounded; success is multidimensional and graded; strategy-to-outcome mapping is learned and adversarial                         | All absent       | Highly variable; no standard measurement |

| Task                       | Domain   | R empirical (best observed)                             | Why Not Boundable                                                                                                                    | Property Missing               | Variance Across Processors                |
|----------------------------|----------|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|-------------------------------------------|
| Musical interpretation     | Music    | Unknown (no operation counting standard for expression) | Correctness is aesthetic and culturally graded; input (score) is formally specified but output (performance) is not binary-evaluable | P2 absent (graded correctness) | Enormous; from mechanical to transcendent |
| Emergency triage           | Medicine | 3–5 ops (experienced triage nurse)                      | Patient presentation space unbounded; severity is continuous not categorical; per-assessment information varies by presentation      | P1, P2, P3 absent              | 3–15 ops; $5 \times$ range                |
| Startup strategy selection | Business | Unknown                                                 | Market dynamics not formally characterizable; success is graded and delayed; information per action is highly uncertain              | All absent                     | Unmeasurable by current methods           |

| Task                          | R empirical (best Domain observed) | Why Not Boundable                                                                                                                   | Property Missing  | Variance Across Processors                    |
|-------------------------------|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-------------------|-----------------------------------------------|
| Foreign language conversation | Variable by complexity             | Comprehension is graded; context is unbounded; per-word information varies by speaker's compression ratio for receiver's vocabulary | P1, P2, P3 absent | Varies by proficiency level and language pair |

**Table F: Structural Property Analysis**

| Property                   | What It Enables                                                             | How to Test For It                                                                                             | When Absent                                                                        | Domains Where Typically Present                                                           | Domains Where Typically Absent                                                   |
|----------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| P1: Enumerable input space | Information-theoretic lower bounds; counting arguments; entropy computation | Can you define a set containing all possible inputs? Can you count the set or compute its information content? | No lower bound from input structure; cannot compute minimum information extraction | Computation (formal inputs); mathematics (formal problems); manufacturing (defined parts) | Medicine (unbounded presentations); combat (unbounded situations); creative arts |

| Property                                             | What It Enables                                                                                              | How to Test For It                                                                                                                           | When Absent                                                                                                                                                                                                                    | Domains Where Typically Present                                                                                                   | Domains Where Typically Absent                                                                                |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| P2:<br>Decid-<br>able<br>cor-<br>rect-<br>ness       | Adversary<br>argu-<br>ments;<br>verification-<br>based<br>lower<br>bounds;<br>crisp $R^*$<br>defini-<br>tion | Given<br>input and<br>output, can<br>a finite<br>procedure<br>determine<br>if the<br>output is<br>correct?                                   | $R^*$ def-<br>inition<br>is im-<br>precise<br>(mini-<br>mum<br>for “ap-<br>proxi-<br>mately<br>correct”<br>vs<br>“exactly<br>cor-<br>rect”);<br>adver-<br>sary<br>cannot<br>distin-<br>guish<br>correct<br>from in-<br>correct | Computation<br>(verifiable<br>outputs);<br>mathematics<br>(verifiable proofs);<br>manufacturing<br>(measurable<br>specifications) | Medicine (graded<br>diagnoses); arts<br>(aesthetic quality);<br>business<br>(multidimensional<br>success)     |
| P3:<br>Bounded<br>infor-<br>ma-<br>tion<br>per<br>op | Division<br>argument<br>(total<br>informa-<br>tion /<br>per-op<br>bound =<br>mini-<br>mum<br>ops)            | Is there a<br>maximum<br>amount of<br>informa-<br>tion any<br>single<br>operation<br>can extract,<br>regardless<br>of who<br>performs<br>it? | Experts<br>may<br>extract<br>more<br>per op<br>than<br>novices;<br>floor<br>de-<br>pends<br>on pro-<br>cessor<br>capabil-<br>ity, not<br>just<br>task<br>struc-<br>ture                                                        | Computation (bit<br>per comparison);<br>formal protocols<br>(one check per<br>item)                                               | Medicine (expert<br>glance vs novice<br>examination);<br>aviation<br>(experienced scan<br>vs novice fixation) |

|                                             | What It<br>Property Enables                                                                             | How to<br>Test For It                                                                                           | When<br>Absent                                                                                                                                                                                                     | Domains Where<br>Typically Present                                                                    | Domains Where<br>Typically Absent                                                              |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| P4:<br>Con-<br>structible<br>adver-<br>sary | Tight<br>lower<br>worst-<br>case<br>analysis;<br>proof<br>that floor<br>applies to<br>all<br>strategies | Can you<br>construct<br>an input<br>that forces<br>any correct<br>processor<br>to use at<br>least $R^*$<br>ops? | Lower<br>bound<br>may<br>hold on<br>average<br>but not<br>worst<br>case, or<br>vice<br>versa;<br>cannot<br>rule out<br>clever<br>strate-<br>gies<br>that<br>circum-<br>vent the<br>appar-<br>ent<br>lower<br>bound | Computation<br>(well-defined<br>adversary games);<br>mathematics<br>(constructive<br>counterexamples) | Medicine (no<br>adversarial<br>patient); social<br>domains (no<br>constructible<br>worst case) |

**Table G: Class Transitions — Historical**

| Task                                                      | Domain   | Original<br>Class | Current<br>Class | Transition<br>Trigger              | Year<br>(approximate) | Mechanism                                                                        |
|-----------------------------------------------------------|----------|-------------------|------------------|------------------------------------|-----------------------|----------------------------------------------------------------------------------|
| Myocardial<br>infarc-<br>tion<br>di-<br>ag-<br>no-<br>sis | Medicine | E                 | B                | Troponin<br>biomarker<br>discovery | 1990s                 | Added<br>formal<br>crite-<br>rion;<br>estab-<br>lished<br>mini-<br>mum<br>checks |

| Task                             | Domain      | Original Class | Current Class | Transition Trigger                   | Year (approximate) | Mechanism                                                        |
|----------------------------------|-------------|----------------|---------------|--------------------------------------|--------------------|------------------------------------------------------------------|
| Diabetes diagnosis               | Medicine    | E              | B             | Hemoglobin A1c threshold established | 2010               | Added quantitative criterion with defined threshold              |
| Chess endgame ( $\leq 7$ pieces) | Game theory | B              | P             | Exhaustive tablebase computation     | 2012               | Computed exact $R^*$ for all covered positions                   |
| Checkers (complete game)         | Game theory | B              | P             | Complete solution computed           | 2007               | Exhaustive backward induction; proved optimal play               |
| Protein structure prediction     | Biology     | E              | B             | AlphaFold architecture               | 2020               | Established computational upper bound; physics gives lower bound |
| Arithmetic operations            | Mathematics | E              | P             | Positional notation with zero        | ~500 CE            | Formalized task structure; made operation count derivable        |

| Task                             | Domain        | Original Class | Current Class    | Transition Trigger                                 | Year (approximate) | Mechanism                                                         |
|----------------------------------|---------------|----------------|------------------|----------------------------------------------------|--------------------|-------------------------------------------------------------------|
| Sorting                          | Computational | NP (always)    | P                | Information-theoretic proof                        | 1960s              | Task was always Class P; proof was discovered, not created        |
| Traveling salesman (approximate) | Computational | NP (wide gap)  | B (narrower gap) | Christofides algorithm; LKH heuristic improvements | 1976–present       | Better algorithms narrowed gap without closing it                 |
| DNA sequencing cost              | Biology       | E              | B                | Next-gen sequencing established throughput bounds  | 2000s              | Technology created formal throughput floor and achievable ceiling |
| Disease classification (various) | Medicine      | E              | B                | ICD coding criteria; diagnostic scoring systems    | 1900s–present      | Formal criteria created; minimum check counts computable          |

| Task                                | Domain        | Original Class | Current Class | Transition Trigger                                            | Year (approximate) | Mechanism                                                                                      |
|-------------------------------------|---------------|----------------|---------------|---------------------------------------------------------------|--------------------|------------------------------------------------------------------------------------------------|
| Language trans-<br>la-<br>tion      | Communication | C              | B             | Statistical/neural MT established<br>BLEU score bounds        | 2000s–present      | Formal quality metrics created measurable bounds; still wide gap                               |
| Image clas-<br>sifi-<br>ca-<br>tion | Computation   | C              | B             | Information-theoretic limits on accuracy vs sample complexity | 2010s              | PAC learning bounds established structural floor; deep learning established ceiling            |
| Factoring large integers            | Computation   | C (wide gap)   | B (narrowing) | Number field sieve; quantum algorithms (theoretical)          | 1990s–present      | Better algorithms; Shor’s algorithm gives polynomial upper bound if quantum computer available |



| Task               | Domain              | Original Class | Current Class | Transition Trigger                                                         | Year (approximate) | Mechanism                                                                       |
|--------------------|---------------------|----------------|---------------|----------------------------------------------------------------------------|--------------------|---------------------------------------------------------------------------------|
| Weather prediction | Atmospheric science | Eric           | B             | Computational fluid dynamics established resolution-accuracy relationships | 1960s–present      | Physics gives theoretical limits; computational models give achievable accuracy |

**Table H: Domain Classification Summary**

| Domain      | Predominant Class | Class P Tasks (examples)                                                                  | Class B Tasks (examples)                                             | Class E Tasks (examples)                                  | P:B:E Ratio (approximate) |
|-------------|-------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------|---------------------------|
| Computation | Eric              | Sorting, searching, graph traversal, parity, polynomial evaluation                        | NP-hard optimization, matrix multiplication, channel coding          | Heuristic algorithm design, novel problem solving         | 60:30:10                  |
| Mathematics | Mixed             | Specific computations (polynomial evaluation, system solving, integration of known forms) | Optimization problems, approximation theory, proof complexity bounds | Creative proof search, conjecture resolution              | 30:30:40                  |
| Medicine    | Eric              | Algorithmic protocols (ACLS, sepsis screening)                                            | Criteria-based diagnosis (RA, sepsis score), lab panel minimization  | General diagnosis, treatment selection, prognosis, triage | 5:20:75                   |

| Domain                | Predominant Class | Class P Tasks (examples)                                           | Class B Tasks (examples)                                             | Class E Tasks (examples)                                          | P:B:E Ratio (approximate) |
|-----------------------|-------------------|--------------------------------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------|---------------------------|
| Aviation (trans-port) | Mixed             | Checklist execution, navigation computation, fuel calculation      | Approach optimization, separation assurance computation              | Threat assessment, emergency response, crew resource management   | 25:30:45                  |
| Aviation (combat)     | E-heavy           | Weapons employment zone computation, intercept geometry            | Mission planning optimization, fuel management                       | Threat classification, engagement sequencing, tactical deception  | 15:20:65                  |
| Software engineering  | Mixed             | Algorithm implementation (inherits algorithm's class), compilation | Known-pattern bug diagnosis, performance optimization with profiling | Novel bug diagnosis, architecture design, code review             | 20:35:45                  |
| Manufacturing         | E-heavy           | Simple assembly (count physical actions)                           | Complex assembly with alternatives, quality inspection sequencing    | Process innovation, defect root cause for novel failures          | 25:50:25                  |
| Cooking               | E                 | Boiling water (physical minimum: heat to 100°C)                    | Baking (chemical constraints give bounds on time/temperature)        | Recipe optimization, flavor balancing, plating aesthetics         | 5:15:80                   |
| Air traffic control   | Mixed             | Separation calculation, communication protocol                     | Traffic flow optimization, sector capacity analysis                  | Dynamic traffic management, conflict resolution under uncertainty | 15:35:50                  |

| Domain           | Predominant Class | Class P Tasks (examples)                                  | Class B Tasks (examples)                                    | Class E Tasks (examples)                                                     | P:B:E Ratio (approximate) |
|------------------|-------------------|-----------------------------------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------|
| Customer support | E-heavy           | Scripted protocols for known issues                       | Diagnostic trees for known issue categories                 | Novel issue diagnosis, customer de-escalation, cross-product troubleshooting | 10:25:65                  |
| Education        | B-heavy           | Test scoring (count correct answers)                      | Curriculum sequencing (prerequisite structure gives bounds) | Pedagogical strategy, student engagement, assessment design                  | 5:15:80                   |
| Law              | E-heavy           | Procedural filing requirements (count mandated documents) | Evidence evaluation (rules of evidence constrain process)   | Case strategy, argument construction, jury persuasion                        | 5:20:75                   |

**Table I: Classification Procedure**

| Step | Question                                                                     | Yes →                                      | No →                                           | Tool / Method                                                                  |
|------|------------------------------------------------------------------------------|--------------------------------------------|------------------------------------------------|--------------------------------------------------------------------------------|
| 1    | Can you formally define the space of possible inputs?                        | Proceed to Step 2                          | Provisional Class E                            | Attempt formal specification; test for countability, entropy computability     |
| 2    | Is output correctness decidable by finite procedure?                         | Proceed to Step 3                          | Class E (graded/subjective correctness)        | Define verification procedure; test edge cases; check if correctness is binary |
| 3    | Can you compute minimum information extraction from input to correct output? | Lower bound established; proceed to Step 4 | Class B if any structural bound exists; else E | Information-theoretic analysis; counting arguments; adversary construction     |

| Step | Question                                                            | Yes →                                               | No →                                                                            | Tool / Method                                                         |
|------|---------------------------------------------------------------------|-----------------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| 4    | Does a known method achieve the lower bound from Step 3?            | <b>Class P</b> ( $R^*$ = lower bound = upper bound) | Proceed to Step 5                                                               | Algorithmic analysis; compare best known method's cost to lower bound |
| 5    | Does a known method have a quantifiable cost above the lower bound? | <b>Class B</b> (gap = upper – lower; computable)    | <b>Class B</b> with open upper bound (lower bound exists; no tight upper bound) | Upper bound from best known algorithm or procedure                    |

**Confidence annotations:**

| Classification                    | Confidence Level | Revision Conditions                                                    |
|-----------------------------------|------------------|------------------------------------------------------------------------|
| Class P (Step 4 = yes)            | Certain          | Only if proof contains error (should not happen)                       |
| Class B (Step 5 = yes)            | High             | Gap may narrow; classification stable                                  |
| Class B, open upper (Step 5 = no) | Moderate         | Better method may close gap toward P; worse: problem may be deeper E   |
| Class E (Step 1 or 2 = no)        | Provisional      | New structural insight may enable formalization → reclassify to B or P |

**Table J: Implications Matrix**

| Derivability Class | Training Target                                              | Assessment Standard                             | Certification Basis                       | Infrastructure Optimality                                   | Revision Frequency                                    | Expert-Infrastructure Gap                                                             |
|--------------------|--------------------------------------------------------------|-------------------------------------------------|-------------------------------------------|-------------------------------------------------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------|
| P (Provable)       | Exact: $R^*$ (known value)                                   | Absolute: distance from $R^*$ in ops            | Rigorous: demonstrate $R^*$ or better     | Provably optimal: encodes exactly $R^*$ operations          | Never (unless task changes)                           | Zero (infrastructure matches proven optimum)                                          |
| B (Boundable)      | Range: $R$ upper (achievable) toward $R$ lower (theoretical) | Bounded: position within $[R$ lower, $R$ upper] | Bounded: demonstrate $R$ upper or better  | Best-effort: encodes $R$ upper; may include unnecessary ops | When better algorithms or proofs narrow gap           | Small to moderate (expert may find shortcuts within gap)                              |
| E (Empirical)      | Relative: best observed performance                          | Relative: percentile in population distribution | Normative: demonstrate top-N% performance | Empirically calibrated: encodes current best practice       | When better performance observed or practice analyzed | Moderate to large (expert has dissolved optimizations infrastructure hasn't captured) |

**Table K: Meta-Derivability**

| Question                          | Class P Answer                                                                  | Class B Answer                                                                                                | Class E Answer                                                                       | Open Problem?                                |
|-----------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------------|
| How many ops to determine $R^*$ ? | Proof complexity of the lower bound theorem; finite and computable in principle | Sum of proof complexity (lower bound) + algorithmic analysis (upper bound); finite but potentially very large | Undefined or infinite; may require exhaustive observation of all possible processors | Yes: when is meta- $R^*$ finite vs infinite? |

| Question                                        | Class P Answer                                                             | Class B Answer                                                      | Class E Answer                                                              | Open Problem?                                                  |
|-------------------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------|
| Is the class determination itself decidable?    | Yes (verify the proof)                                                     | Partially (verify the bounds; gap computability may be undecidable) | Unknown (no finite procedure to prove that no structural bound exists)      | Yes: connects to halting problem and Gödel's incompleteness    |
| Can a task's class be determined automatically? | Yes for some (automated theorem proving can verify known proof structures) | Partially (automated analysis for known bound techniques)           | No in general (would require proving a negative — that no structure exists) | Yes: automated classification is itself a Class B or E problem |
| What is the $R^*$ of determining $R^*$ ?        | Meta- $R^*$ exists and is finite (proportional to proof complexity)        | Meta- $R^*$ bounded (between cost of finding both bounds)           | Meta- $R^*$ may be unbounded or undefined                                   | Yes: self-referential; connects to foundations of mathematics  |

**Table L: Complexity Class Parallel**

| Derivability Property                           | Derivability Class | Complexity Parallel              | Structural Similarity                                                           | Structural Difference                                                                     |
|-------------------------------------------------|--------------------|----------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Answer efficiently computable from structure    | P (Provable)       | P (Polynomial time)              | Both: answer derivable by efficient procedure from input                        | Derivability P: one answer per task ( $R^*$ ).<br>Complexity P: answer per input instance |
| Answer verifiable but not efficiently derivable | B (Boundable)      | NP (Nondeterministic Polynomial) | Both: given a claimed answer, verification is feasible; finding optimal is hard | Derivability B: bounds from structure.<br>Complexity NP: verification from witness        |

| Derivability Property             | Derivability Class                               | Complexity Parallel                         | Structural Similarity                                                           | Structural Difference                                                                                    |
|-----------------------------------|--------------------------------------------------|---------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Answer structurally inaccessible  | E (Empirical)                                    | Undecidable                                 | Both: no general algorithm produces the answer                                  | Derivability E: answer exists but isn't derivable.<br>Undecidable: answer may not exist in formal system |
| Answer gap is the central mystery | B gap (R upper – R lower)                        | P vs NP question                            | Both: the question of whether the gap can be closed is the deepest open problem | Derivability gap: per-task.<br>P vs NP: for an entire class of problems                                  |
| Historical progress narrows gap   | $B \rightarrow P$ transitions                    | Complexity class separations                | Both: structural discoveries move tasks toward more complete understanding      | Derivability: transitions are common and productive.<br>Complexity: separations are rare and monumental  |
| Hierarchy is directional          | $E \rightarrow B \rightarrow P$ (never reverses) | Results are permanent (proofs don't expire) | Both: proven results persist; knowledge accumulates                             | Derivability: task can change, invalidating proof.<br>Complexity: problem definition is fixed            |

**Table M: Prediction Testing Specifications**

| Prediction                                                     | Independent Variable                                            | Dependent Variable                               | Measurement                                                                           | Expected Result                                                        | Falsification                                                                           |
|----------------------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Class P tasks dissolve faster than Class E tasks               | Derivability class (P vs E) of task                             | Dissolution curve rate parameter $\lambda$       | Measure dissolution curves for matched-difficulty tasks across classes; compare rates | Class P tasks have higher $\lambda$ (faster dissolution toward $R^*$ ) | No significant difference in $\lambda$ between classes after controlling for difficulty |
| Class P infrastructure closes expert-novice gap completely     | Derivability class; presence of provably optimal infrastructure | Expert-novice gap with vs without infrastructure | Measure novice performance with and without Class P infrastructure; compare to expert | Novice with Class P infrastructure matches expert $R^*$                | Novice with infrastructure still significantly above $R^*$                              |
| Class E infrastructure leaves residual expert-novice gap       | Derivability class; presence of best-practice infrastructure    | Expert-novice gap with infrastructure            | Measure novice with Class E infrastructure vs expert without                          | Expert outperforms infrastructure-aided novice by measurable margin    | Infrastructure-aided novice matches expert                                              |
| Class B gap narrows correlate with $R^*$ precision in training | Bound gap width                                                 | Training efficiency (time to reach $R$ upper)    | Compare training outcomes for tasks with narrow vs wide B-gaps                        | Narrow gap $\rightarrow$ more efficient training (clearer target)      | No correlation between gap width and training efficiency                                |



| Prediction                                                       | Independent Variable      | Dependent Variable                                                     | Measurement                                                                      | Expected Result                                                   | Falsification                                   |
|------------------------------------------------------------------|---------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------|-------------------------------------------------|
| E → B transitions improve training outcomes                      | Pre/post transition class | Training efficiency; assessment reliability                            | Compare training before and after structural discovery changes class from E to B | Post-transition training more efficient; assessment more reliable | No measurable improvement after transition      |
| Class E R* estimates have higher revision frequency than Class B | Derivability class        | Frequency of R* estimate revision (improved best-observed performance) | Track R* estimates over time across tasks of known class                         | Class E R* revises more frequently                                | No significant difference in revision frequency |

**Table N: Specification Summary**

| Metric                                   | Count                                                               |
|------------------------------------------|---------------------------------------------------------------------|
| Formal definitions                       | 12                                                                  |
| Derivability classes defined             | 3 (Provable, Boundable, Empirical)                                  |
| Structural properties identified         | 4 (P1–P4) for Class P; 3 (B1–B3) for Class B; 4 (E1–E4) for Class E |
| Class P examples analyzed                | 8                                                                   |
| Class B examples analyzed                | 10                                                                  |
| Class E examples analyzed                | 10                                                                  |
| Historical class transitions documented  | 14                                                                  |
| Domains classified                       | 12                                                                  |
| Classification procedure steps           | 5                                                                   |
| Training/assessment implications derived | 3 (one per class)                                                   |
| Complexity class parallels drawn         | 6                                                                   |
| Meta-derivability questions posed        | 4                                                                   |
| Testable predictions                     | 6                                                                   |
| Open problems                            | 6                                                                   |

---

## Key Equations Summary

### Optimal reduction definition:

$R^* = \min \text{ ops for correct reliable execution of task}$

### Class P criterion:

$\exists \text{ proof: } \forall \text{ correct processors } p, \text{ ops}(p, \text{ task}) \geq R; R \text{ is exact}$

### Class B criterion:

$\exists \text{ proof: } R_{\text{lower}} \leq R^* \leq R_{\text{upper}}; \text{ gap} = R_{\text{upper}} - R_{\text{lower}} > 0$

### Class E criterion:

$R^* = \min \text{ observed ops}(p, \text{ task}) \text{ across all observed processors } p; \text{ no structural proof}$

### Information-theoretic lower bound (when applicable):

$R^* \geq \lceil \log_2(\text{input space}) / \text{bits\_per\_op} \rceil$

### Adversary lower bound (when applicable):

$R^* \geq \min \text{ ops adversary can force on any correct strategy}$

### Bound gap:

$\text{gap}(\text{task}) = R_{\text{upper}} - R_{\text{lower}}$  (Class B only; zero for P; undefined for E)

### Class hierarchy:

$P \subset B$  (every proven  $R^*$  is trivially a tight bound)

B and E overlap in practice (empirical estimates exist for B tasks too)

$E \rightarrow B \rightarrow P$  transitions driven by structural discovery; never reverse

### Fundamental inequality (from prior work, referenced throughout):

$\Sigma \text{ ops} \times \bar{d} \leq N$

---

*HOWL-MATH-20-2026. Derivability Classes of Optimal Reduction: The Derivability Spectrum from Proof to Measurement.*

[[HOWL-MATH-20-2026](#)] Derivability Classes of Optimal Reduction. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-20-2026>

[[HOWL-INFO-11-2026](#)] The Relationship of Zero, One, and Infinity in Information Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-11-2026>

[[HOWL-INFO-12-2026](#)] Information Processing Requires Reduction to Cardinality One. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-12-2026>

[[HOWL-INFO-13-2026](#)] The Six States of Information. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-13-2026>

[[HOWL-MATH-14-2026](#)] A Mathematical Theory of Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-14-2026>

[[HOWL-MATH-15-2026](#)] A Measurement Theory of Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-15-2026>

[[HOWL-MATH-16-2026](#)] The Geometry of Dissolution and Fragility. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-16-2026>

[[HOWL-MATH-17-2026](#)] Processing Entropy as a Metric Space. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-17-2026>

[[HOWL-MATH-18-2026](#)] Concurrency Tax from System Structure. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-18-2026>

[[HOWL-MATH-19-2026](#)] The Mathematics of Processing-Aware Communication. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-19-2026>